МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение

высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных технологий

Отчет по практической работе №3

по дисциплине «Технологии разработки программных приложений»

Тема практической работы: «Docker»

Выполнил:
Студент группы ИНБО-22-24 Пчелкин Александр Сергеевич

Проверил: Доцент кафедры МОСИТ,
кандидат экономических наук,
доцент Исабекова О.А.

Москва 2026

Цель практической работы: изучение основных возможностей Docker, включая работу с образами, контейнерами, портами, томами, переменными окружения и создание Dockerfile, приобретение навыков сборки и запуска контейнеров, а также выполнение индивидуального задания с учетом специфики варианта.

Ход работы

3.1.1. Образы

```
C:\Users\APche>docker images

IMAGE   ID             DISK USAGE   CONTENT SIZE   EXTRA

C:\Users\APche>docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
b40150c1c271: Pull complete
92842f25412d: Download complete
Digest: sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc41c7b
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
```

Рисунок 1 — посмотрел имеющиеся образы и загружаем новый

```
C:\Users\APche>docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Рисунок 2 — посмотрел контейнеры

3.1.2 Изоляция

```
C:\Users\APche>hostname
Home-PC

C:\Users\APche>hostname
Home-PC

C:\Users\APche>docker run ubuntu hostname
ffe4c7d459ba

C:\Users\APche>docker run ubuntu hostname
ea28279aa7b5
```

Рисунок 3 — получил разный hostname для 2 контейнеров созданных из 1 образа

```
C:\Users\APche>docker ps -a
CONTAINER ID   IMAGE     COMMAND      CREATED         STATUS                      PORTS     NAMES
ea28279aa7b5   ubuntu    "hostname"   2 minutes ago   Exited (0) 2 minutes ago              quizzical_cannon
ffe4c7d459ba   ubuntu    "hostname"   2 minutes ago   Exited (0) 2 minutes ago              hungry_chatelet

C:\Users\APche>docker run ubuntu bash
```

Рисунок 4 — проверил контейнеры и запустили bash

```
C:\Users\APche>docker run -it ubuntu bash
root@23a68906ff6b:/#
```

Рисунок 5 — запустил bash

3.1.3. Работа с портами

```
C:\Users\APche>docker pull python
Using default tag: latest
latest: Pulling from library/python
3b32e3bb7338: Pull complete
10ef5c83b2fa: Pull complete
eeb7fd4cccd4: Pull complete
5467f93954cf: Pull complete
20d12c8f3f3f: Pull complete
1e1bb20756a8: Pull complete
bbfdb8accfe4: Pull complete
98628e8e6ba5: Download complete
fe4f0780b2ba: Download complete
Digest: sha256:0ba001803c72c128063cfa88863755f905cefabe73c026c66a5a86d8f1d63e98
Status: Downloaded newer image for python:latest
docker.io/library/python:latest
```

Рисунок 6 — загрузили образ python

```
C:\Users\APche>docker run -it python python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Рисунок 7 — Запустили модуль веб-сервера

Запустил модуль веб-сервера, при запуске пишется, что сервер доступен по адресу http://0.0.0.0:8000/. Однако, если открыть этот адрес, то ничего не будет видно, потому что порты не проброшены. Port Forwarding — термин, который означает «переадресация портов», «проброс портов», «перенаправление портов» или «трансляция портов». Это механизм, при котором трафик, приходящий на определённый порт внешнего IP-адреса, перенаправляется на конкретное устройство и порт во внутренней сети.

```
C:\Users\APche>docker run -it python python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
^C
Keyboard interrupt received, exiting.

C:\Users\APche>docker run -it -p8000:8000 python python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
172.17.0.1 - - [30/Apr/2026 13:24:39] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [30/Apr/2026 13:24:39] code 404, message File not found
172.17.0.1 - - [30/Apr/2026 13:24:39] "GET /favicon.ico HTTP/1.1" 404 -
```

Рисунок 8 — Запустили модуль веб-сервера с пробросом портов

3.1.4. Именованные контейнеры, остановка и удаление

```
C:\Users\APche>docker run -p8000:8000 --name pyserver -d python python -m http.server
14e565a2a57df21c24a5989de41d0d116f80f38d509924658485b76c3b36aebd

C:\Users\APche>docker ps | findstr "pyserver"
14e565a2a57d   python    "python -m http.serv…"   26 seconds ago   Up 25 seconds   0.0.0.0
:8000->8000/tcp, [::]:8000->8000/tcp   pyserver
```

Рисунок 9 — запустил контейнер в фоне и проверил работает ли он

```
C:\Users\APche>docker logs pyserver

C:\Users\APche>docker stop pyserver
pyserver
```

Рисунок 10 —запустил команду просмотра логов и остановил контейнер

```
C:\Users\APche>docker run -it -p8000:8000 --name pyserver -d python python -m

What's next:
    Debug this container error with Gordon → docker ai "help me fix this container error"
docker: Error response from daemon: Conflict. The container name "/pyserver" is already in
 use by container "14e565a2a57df21c24a5989de41d0d116f80f38d509924658485b76c3b36aebd". You
have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
```

Рисунок 11 — ошибка

При создании контейнера с уже занятым именем вылетает ошибка для этого удалил контейнер с этим именем

```
C:\Users\APche>docker rm pyserver
pyserver
```

Рисунок 12 — удалил контейнер pyserver

3.1.5. Постоянное хранение данных

“docker run -p8000:8000 --name pyserver –-rm -d python python -m http.ser ver -d /mnt” разберем параметры следующей команды. Запускаем контейнер н а внешнем порте 8000 и на внутреннем 8000(docker run -p8000:8000). Присва иваем имя(--name) контейнеру pyserver. Контейнер удалится после окончания работы (–-rm). Запускаем контейнер в фоне(-d/--detach). Используем образ pyt hon (1 python). Используем модуль веб сервера из пайтон (2 python, -m говори т не запускай py файл, а выполни модуль http.server). Флаг -d /mnt указывает модулю http.server, какая директория будет корневой для отображения.

```
:\Users\APche>docker run -p 8000:8000 --name pyserver --rm -d python python -m http.server -d /mnt
e0f44faf8f03f5b1cf5a0a3330cf159acd8505b94f67743fa7b02b4657e9a5b8
```

Рисунок 13 — Запустили контейнер, в котором веб-сервер будет отдавать содержимое директории /mnt

```
C:\Users\APche>docker exec -it pyserver bash
root@e0f44faf8f03:/# cd mnt && echo "hello world" > hi.txt
root@e0f44faf8f03:/mnt#
exit
```

Рисунок 14 — Зашли в контейнер записали в файл hello world

Удалил контейнер и запустил заново с помощью команды с рисунка 13.

3.1.6. Тома

```
:\Users\APche>docker run -p8000:8000 --rm --name pyserver -d -v myVolume:/mnt python python -m http.server -d /mnt
4b91b958cd9a6e34a6790fa7ba4333a94a75924b57a26c1a67affa27234f8bf

:\Users\APche>docker exec -it pyserver bash
oot@f4b91b958cd9:/# cd mnt && echo "hello world" > hi.txt
oot@f4b91b958cd9:/mnt# docker inspect -f "{{json .Mounts}}" pyserver^C
oot@f4b91b958cd9:/mnt#
xit

hat's next:
   Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug pyserver
   Learn more at https://docs.docker.com/go/debug-cli/

:\Users\APche>docker inspect -f "{{json .Mounts}}" pyserver
{"Type":"volume","Name":"myVolume","Source":"/var/lib/docker/volumes/myVolume/_data","Destination":"/mnt","Driver":"loc
l","Mode":"z","RW":true,"Propagation":""}]
```

Рисунок 15 — 3.1.6

Запустил контейнер с примонтированным томом, записал в него данные и потом узнал где хранятся данные.

3.1.7. Монтирование директорий и файлов

```
C:\Users\APche\myfiles>echo. > host.txt

C:\Users\APche\myfiles>docker run -p 8000:8000 --rm --name pyserver -d -v C:\Users\APche\myfiles:/mnt pytho
n python -m http.server -d /mnt
a5b6259b99871c5afcc09de78b6cfc73564db8945e14d46c1b493532bc2ada19

C:\Users\APche\myfiles>docker exec -it pyserver bash
root@a5b6259b9987:/# cd /mnt
root@a5b6259b9987:/mnt# ls
host.txt
root@a5b6259b9987:/mnt# echo "hello world" > hi.txt
root@a5b6259b9987:/mnt#
exit
```

Рисунок 16 — пробросил папку myfiles в папку /mnt

```
:\Users\APche\myfiles>docker run -p 8000:8000 --rm --name pyserver -d -v C:\Users\APche\myfiles\host.txt:/
nt/nhost.txt python python -m http.server -d /mnt
352b5cdfdc8f5b5372262a0a8cce8f9b96a64d42756be37a4b4e02ceb5d2fac7
```

Рисунок 17 — пробросил файл из папки myfiles в папку /mnt

3.1.8. Переменные окружения

```
C:\Users\APche>docker run -it --rm -e MIREA="ONE LOVE" ubuntu env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=95d58995f3b3
TERM=xterm
MIREA=ONE LOVE
HOME=/root
```

Рисунок 18 — передал переменную окружения внутрь контейнера

3.1.9. Dockerfile

Используя dockerfile собрал образ

```
ПРОВОДНИК                    Приветствие    dockerfile ×
DOCKER                       dockerfile
> data
dockerfile
1  FROM ubuntu:22.04
2  RUN apt update \
3  && apt install -y python3 fortune \
4  && cd /usr/bin \
5  && ln -s python3 python
6  RUN /usr/games/fortune > /mnt/greeting-while-building.txt
7  ADD ./data /mnt/data
8  EXPOSE 80
9  CMD ["python", "-m", "http.server", "-d", "/mnt/", "80"]
```

Рисунок 18 — dockerfile в vs code

```
C:\Users\APche\Documents\МИРЭА\Technologies_development\docker>docker build -t mycoolimage .
[+] Building 321.1s (9/9) FINISHED                                          docker:desktop-linux
```

Рисунок 19 — успешная сборка образа

```
C:\Users\APche\Documents\МИРЭА\Technologies development\docker>docker run --rm -it -p8099:80 mycoolimage
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Рисунок 20 — успешный запуск контейнера

3.2. Практическая часть

Вариант 17 порт 8817 пакет figlet.

```
DOCKER
data
student.txt
dockerfile

dockerfile
FROM ubuntu:24.04
RUN apt update \
    && apt install -y python3 figlet \
    && cd /usr/bin \
    && ln -s python3 python
ADD ./data /mnt/files
EXPOSE 8817
CMD ["python", "-m", "http://server", "-d", "/mnt/files", "8817"]

ПРОБЛЕМЫ   ВЫХОДНЫЕ ДАННЫЕ   КОНСОЛЬ ОТЛАДКИ   ТЕРМИНАЛ   ПОРТЫ   cmd

C:\Users\APche\Documents\МИРЭА\Technologies_development\docker>docker build -t mycoolimage .
[+] Building 17.3s (8/8) FINISHED                                          docker:desktop-li
 => [internal] load build definition from dockerfile
 => => transferring dockerfile: 266B
 => [internal] load metadata for docker.io/library/ubuntu:24.04
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/3] FROM docker.io/library/ubuntu:24.04@sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfc
 => => resolve docker.io/library/ubuntu:24.04@sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfc
 => [internal] load build context
 => => transferring context: 129B
 => [2/3] RUN apt update     && apt install -y python3 figlet     && cd /usr/bin     && ln -s pytho
 => [3/3] ADD ./data /mnt/files
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:9c884c4f68033f07768fc69e8ef11d010437dc7332c0ba4ef6c221b67eb78773
 => => exporting config sha256:a858aec580d47f92f9c70eda53367b7ed166baa0c8a5569e2482876a1f7975a4
 => => exporting attestation manifest sha256:7352dd9c0724d11ad860105c6a67272455cb4c48cba5e6c94c16bce
 => => exporting manifest list sha256:2c7f0dd73e7f306b80fea5487ffb5d9327f66e2e5055262b67ea24427a4fea
 => => naming to docker.io/library/mycoolimage:latest
 => => unpacking to docker.io/library/mycoolimage:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/iniz16i5d69mmzudkt3cuh
```

Рисунок 21 — собрал образ из своего докерфайла

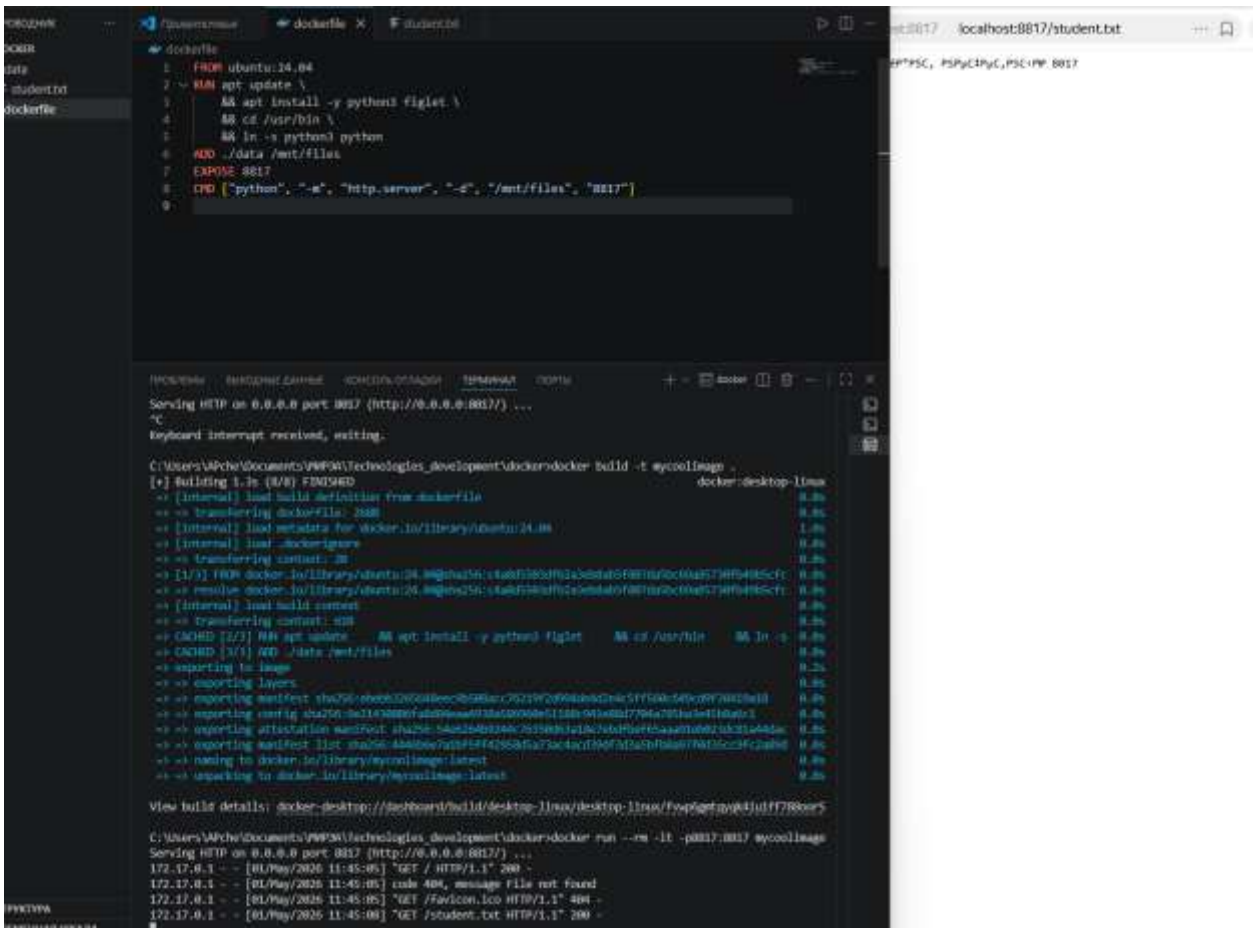

Рисунок 22 — запустил образ и проверил в браузере текст

Directory listing for /

- student.txt

Рисунок 23 — запустил образ и проверил в браузере текст есть

Вывод

В ходе практической работы были изучены основные принципы работы с Docker:

Контейнеризация vs виртуализация. Контейнеры используют ядро хостовой системы и изолируют только пользовательское пространство, в отличие от виртуальных машин.

Проброс портов (-p). Флаг -p hostPort:containerPort создаёт сетевой туннель между хостом и контейнером. Без него приложение внутри контейнера недоступно снаружи. Изучена разница между EXPOSE (документирование) и -p (фактический проброс).

Монтирование директорий и файлов (-v) позволяет связать папку хоста с папкой внутри контейнера. Изменения видны мгновенно в обе стороны.

Переменные окружения (-e). Флаг -e передаёт переменные окружения с хоста внутрь контейнера, не сохраняя их в образе. Удобно для паролей и настроек.

Dockerfile и сборка образа. Изучены команды FROM, RUN (сборка), ADD (копирование файлов), EXPOSE (метаданные порта), CMD. Собран образ на базе ubuntu:24.04 с Python и figlet, запущен веб-сервер на порту 8817 с содержимым примонтированной папки.